Lab 2 - Bangun Indeks & Cari

ukuran chunk 1000, overlap 15%, backend: lsa

Langkah 1. Membangun indeks

Tiga PDF diletakkan di folder dokumen/. Setiap file melaporkan jumlah chunk-nya:

File	Jumlah chunk
jaringan_saraf_deep_learning.pdf	4
nlp_model_bahasa_besar.pdf	4
sejarah_kecerdasan_buatan.pdf	4
Total	12

```
jaringan_saraf_deep_learning.pdf: 4 chunk
nlp_model_bahasa_besar.pdf: 4 chunk
sejarah_kecerdasan_buatan.pdf: 4 chunk
Total 12 chunk | ukuran=1000 overlap=15% | backend=lsa
```

Langkah 2. Lima pertanyaan dan pemeriksaan chunk teratas

Sebuah chunk dianggap relevan bila memuat semua kata kunci jawaban. Tabel menunjukkan 3 chunk teratas.

Siapa yang memperkenalkan perceptron dan pada tahun berapa?

Peringkat	Skor	Sumber	Relevan?	Cuplikan
1	0.892	jaringan_saraf_deep_learning.pdf #0	ya	Jaringan Saraf Tiruan dan Deep Learning 1. Neuron Buatan Neuron buatan menerima beberapa masukan, mengalikan tiap masukan dengan bobot, menj...
2	0.464	nlp_model_bahasa_besar.pdf #1	tidak	tihan sering membutuhkan lebih banyak token untuk makna yang sama. 3. Representasi Kata Word2vec yang diperkenalkan Mikolov dan kawan-kawan ...
3	0.437	jaringan_saraf_deep_learning.pdf #3	tidak	mbar Klasifikasi gambar dan deteksi objek RNN/LSTM Data berurutan Prakiraan deret waktu Transformer Teks dan urutan panjang Penerjemahan dan...

Pada tahun berapa AlexNet memenangkan lomba ImageNet?

Peringkat	Skor	Sumber	Relevan?	Cuplikan
1	0.935	sejarah_kecerdasan_buatan.pdf #2	ya	y! melawan dua juara manusia. Pada masa ini, fokus bergeser dari aturan buatan tangan ke pembelajaran dari data. Tersedianya data dalam juml...
2	0.861	sejarah_kecerdasan_buatan.pdf #3	ya	kecerdasan berpikir mesin 1956 Konferensi Dartmouth Kelahiran istilah dan bidang AI 1997 Deep Blue mengalahkan Kasparov Mesin unggul dalam c...
3	0.362	nlp_model_bahasa_besar.pdf #1	tidak	tihan sering membutuhkan lebih banyak token untuk makna yang sama. 3. Representasi Kata Word2vec yang diperkenalkan Mikolov dan kawan-kawan ...

Apa fungsi mekanisme self-attention pada Transformer?

Peringkat	Skor	Sumber	Relevan?	Cuplikan
1	0.861	nlp_model_bahasa_besar.pdf #1	ya	tihan sering membutuhkan lebih banyak token untuk makna yang sama. 3. Representasi Kata Word2vec yang diperkenalkan Mikolov dan kawan-kawan ...
2	0.567	sejarah_kecerdasan_buatan.pdf #3	tidak	kecerdasan berpikir mesin 1956 Konferensi Dartmouth Kelahiran istilah dan bidang AI 1997 Deep Blue mengalahkan Kasparov Mesin unggul dalam c...
3	0.303	jaringan_saraf_deep_learning.pdf #0	tidak	Jaringan Saraf Tiruan dan Deep Learning 1. Neuron Buatan Neuron buatan menerima beberapa masukan, mengalikan tiap masukan dengan bobot, menj...

Apa kelemahan sistem pakar?

Peringkat	Skor	Sumber	Relevan?	Cuplikan
1	0.883	sejarah_kecerdasan_buatan.pdf #0	ya	Sejarah dan Perkembangan Kecerdasan Buatan 1. Awal Mula Pada tahun 1950, Alan Turing menerbitkan makalah Computing Machinery and Intelligenc...
2	0.612	sejarah_kecerdasan_buatan.pdf #1	ya	a kimia. Kelemahan sistem pakar adalah aturan harus ditulis tangan sehingga mahal dibuat dan sulit dirawat. Sistem juga gagal ketika menghad...
3	0.238	nlp_model_bahasa_besar.pdf #1	tidak	tihan sering membutuhkan lebih banyak token untuk makna yang sama. 3. Representasi Kata Word2vec yang diperkenalkan Mikolov dan kawan-kawan ...

Bagaimana cara kerja dropout dalam mengurangi overfitting?

Peringkat	Skor	Sumber	Relevan?	Cuplikan
1	0.965	jaringan_saraf_deep_learning.pdf #3	ya	mbar Klasifikasi gambar dan deteksi objek RNN/LSTM Data berurutan Prakiraan deret waktu Transformer Teks dan urutan panjang Penerjemahan dan...
2	0.291	jaringan_saraf_deep_learning.pdf #0	tidak	Jaringan Saraf Tiruan dan Deep Learning 1. Neuron Buatan Neuron buatan menerima beberapa masukan, mengalikan tiap masukan dengan bobot, menj...
3	0.264	sejarah_kecerdasan_buatan.pdf #0	tidak	Sejarah dan Perkembangan Kecerdasan Buatan 1. Awal Mula Pada tahun 1950, Alan Turing menerbitkan makalah Computing Machinery and Intelligenc...

Ringkasan: chunk peringkat 1 relevan pada 5/5 pertanyaan; ada chunk relevan di 3 teratas pada 5/5.

Langkah 3. Pertanyaan berparafrase

Dua pertanyaan memakai kata yang berbeda dari teks dokumen. Dibandingkan: pencarian semantik (lsa) vs TF-IDF.

Pertanyaan	Catatan	lsa: top1 / top3	TF-IDF: top1 / top3
Bagaimana jaringan saraf bisa mengingat informasi dari bagian awal data berurutan yang panjang?	Teks memakai 'LSTM', 'gerbang', dan 'ketergantungan jarak jauh'; pertanyaan memakai 'mengingat informasi dari bagian awal'.	ya / ya	ya / ya
Mengapa minat dan dana terhadap AI sempat turun drastis pada masa lalu?	Teks memakai 'musim dingin AI', 'menurun', dan 'dipangkas'; pertanyaan memakai 'turun drastis'.	ya / ya	ya / ya

Chunk teratas untuk "Bagaimana jaringan saraf bisa mengingat informasi dari bagian awal data berurutan yang panjang?": jaringan_saraf_deep_learning.pdf #2 (skor 0.915)

```
. Lapisan pooling mengecilkan ukuran peta fitur sehingga komputasi lebih hemat. LeNet karya Yann LeCun
    pada tahun 1998 adalah salah satu CNN awal yang berhasil dipakai untuk mengenali digit tulisan
    tangan. 5. RNN dan LSTM Recurrent Neural Network (RNN) memproses data berurutan, misalnya teks atau
    deret waktu, dengan menyimpan keadaan tersembunyi dari langkah sebelumnya. RNN biasa sulit mengingat
    i...
```

Chunk teratas untuk "Mengapa minat dan dana terhadap AI sempat turun drastis pada masa lalu?": sejarah_kecerdasan_buatan.pdf #1 (skor 0.858)

```
a kimia. Kelemahan sistem pakar adalah aturan harus ditulis tangan sehingga mahal dibuat dan sulit
    dirawat. Sistem juga gagal ketika menghadapi situasi di luar aturan yang telah ditulis. 3. Musim
    Dingin AI Musim dingin AI adalah masa ketika harapan terhadap AI menurun dan dana penelitian
    dipangkas. Laporan Lighthill pada tahun 1973 di Inggris mengkritik kemajuan AI yang dinilai jauh di
    bawah janji...
```

Kesimpulan

Pada 5 pertanyaan langsung, chunk peringkat 1 relevan pada 5/5 pertanyaan dan salah satu dari 3 chunk teratas relevan pada 5/5 pertanyaan. Pada 2 pertanyaan parafrase, backend 'lsa' menemukan chunk relevan di peringkat 1 pada 2/2 dan di 3 teratas pada 2/2, sedangkan pembanding TF-IDF (kecocokan bentuk kata) 2/2 dan 2/2. Pada dua parafrase ini, semantik dan TF-IDF memberi hasil yang sama, jadi tes ini belum bisa membedakan keduanya: pertanyaan masih berbagi akar kata dengan teks dan TF-IDF n-gram karakter menangkap variasi bentuk kata. Catatan: backend LSA hanya semantik ringan (hubungan antar kata dipelajari dari korpus yang kecil), sehingga parafrase yang benar-benar tidak berbagi kata akan sulit. Untuk hasil lebih kuat, pasang sentence-transformers; kode ini otomatis memakainya.

Lampiran: kode lab2_indeks_cari.py

```
"""Lab 2 - Bangun indeks & cari (25 menit).

1. Letakkan 2-3 PDF di folder dokumen/, jalankan indeks, pastikan tiap file melaporkan jumlah chunk.
2. Ajukan 5 pertanyaan dengan query(); periksa apakah chunk teratas benar-benar relevan.
3. Ajukan 2 pertanyaan berparafrase; apakah pencarian semantik tetap menemukannya?
"""
import io
import os
import sys
from contextlib import redirect_stdout

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from laporan_utils import Laporan
from rag_utils import bangun_indeks, cocok, query

BASE = os.path.dirname(os.path.abspath(__file__))
UKURAN, OVERLAP = 1000, 0.15

# kw = kata kunci yang HARUS ada di chunk teratas agar dianggap relevan
PERTANYAAN = [
    {"q": "Siapa yang memperkenalkan perceptron dan pada tahun berapa?", "kw": ["Rosenblatt", "1958"]},
    {"q": "Pada tahun berapa AlexNet memenangkan lomba ImageNet?", "kw": ["AlexNet", "2012"]},
    {"q": "Apa fungsi mekanisme self-attention pada Transformer?", "kw": ["self-attention", "token"]},
    {"q": "Apa kelemahan sistem pakar?", "kw": ["sistem pakar", "ditulis tangan"]},
    {"q": "Bagaimana cara kerja dropout dalam mengurangi overfitting?", "kw": ["Dropout", "acak"]},
]
PARAFRASE = [
    {"q": "Bagaimana jaringan saraf bisa mengingat informasi dari bagian awal data berurutan yang
    panjang?",
     "kw": ["Long Short-Term Memory"],
     "catatan": "Teks memakai 'LSTM', 'gerbang', dan 'ketergantungan jarak jauh'; pertanyaan memakai
    'mengingat informasi dari bagian awal'."},
    {"q": "Mengapa minat dan dana terhadap AI sempat turun drastis pada masa lalu?", "kw": ["musim dingin
     AI"],
     "catatan": "Teks memakai 'musim dingin AI', 'menurun', dan 'dipangkas'; pertanyaan memakai 'turun
    drastis'."},
]


def ringkas(t, n=150):
    t = " ".join(t.split())
    return t if len(t) <= n else t[:n] + "..."


def jalankan(indeks, daftar, k=3):
    hasil = []
    for s in daftar:
        atas = query(indeks, s["q"], k)
        rel = [cocok(a["teks"], s["kw"]) for a in atas]
        hasil.append((s, atas, rel))
    return hasil


def main():
    # ---- Langkah 1
    print("=== Langkah 1: membangun indeks ===")
    buf = io.StringIO()
    with redirect_stdout(buf):
        idx = bangun_indeks(ukuran=UKURAN, overlap=OVERLAP, backend="auto")
    log_indeks = buf.getvalue()
    print(log_indeks)
    idx_leksikal = bangun_indeks(ukuran=UKURAN, overlap=OVERLAP, backend="tfidf", verbose=False)
    per_file = {}
    for m in idx.meta:
        per_file[m["file"]] = per_file.get(m["file"], 0) + 1

    # ---- Langkah 2
    print("=== Langkah 2: 5 pertanyaan ===")
    h2 = jalankan(idx, PERTANYAAN)
    for s, atas, rel in h2:
        print(f"\nQ: {s['q']}  (kata kunci: {s['kw']})")
        for r, (a, ok) in enumerate(zip(atas, rel), 1):
            print(f"  #{r} skor={a['skor']:.3f} {a['file']}#{a['no']} relevan={ok} |
    {ringkas(a['teks'])}")
    n_top1 = sum(rel[0] for _, _, rel in h2)
    n_top3 = sum(any(rel) for _, _, rel in h2)

    # ---- Langkah 3
    print("\n=== Langkah 3: parafrase ===")
    sem = jalankan(idx, PARAFRASE)
```

```
    lek = jalankan(idx_leksikal, PARAFRASE)
    for (s, a1, r1), (_, a2, r2) in zip(sem, lek):
        print(f"\nQ: {s['q']}")
        print(f"  {idx.backend}: top1 relevan={r1[0]} top3 relevan={any(r1)} |
    {a1[0]['file']}#{a1[0]['no']}")
        print(f"  tfidf : top1 relevan={r2[0]} top3 relevan={any(r2)} | {a2[0]['file']}#{a2[0]['no']}")

    # ---- Kesimpulan dari angka
    sem_top3 = sum(any(r) for _, _, r in sem)
    lek_top3 = sum(any(r) for _, _, r in lek)
    sem_top1 = sum(r[0] for _, _, r in sem)
    lek_top1 = sum(r[0] for _, _, r in lek)
    kes = (f"Pada 5 pertanyaan langsung, chunk peringkat 1 relevan pada {n_top1}/5 pertanyaan dan salah
    satu dari "
           f"3 chunk teratas relevan pada {n_top3}/5 pertanyaan. Pada 2 pertanyaan parafrase, backend "
           f"'{idx.backend}' menemukan chunk relevan di peringkat 1 pada {sem_top1}/2 dan di 3 teratas
    pada "
           f"{sem_top3}/2, sedangkan pembanding TF-IDF (kecocokan bentuk kata) {lek_top1}/2 dan
    {lek_top3}/2. ")
    gagal_top1 = [s["q"] for s, atas, rel in h2 if not rel[0]]
    if gagal_top1:
        kes += ("Pertanyaan yang chunk peringkat 1-nya belum relevan: " + "; ".join(f'"{q}"' for q in
    gagal_top1) +
                ". Penyebab yang paling mungkin: kalimat jawaban terbelah di batas chunk sehingga kata
    kuncinya "
                "terpisah, sementara chunk di sebelahnya memuat topik yang mirip. ")
    if sem_top1 == lek_top1 and sem_top3 == lek_top3:
        kes += ("Pada dua parafrase ini, semantik dan TF-IDF memberi hasil yang sama, jadi tes ini belum
    bisa "
                "membedakan keduanya: pertanyaan masih berbagi akar kata dengan teks "
                "dan TF-IDF n-gram karakter menangkap variasi bentuk kata. ")
    if idx.backend == "lsa":
        kes += ("Catatan: backend LSA hanya semantik ringan (hubungan antar kata dipelajari dari korpus
    yang kecil), "
                "sehingga parafrase yang benar-benar tidak berbagi kata akan sulit. Untuk hasil lebih
    kuat, pasang "
                "sentence-transformers; kode ini otomatis memakainya.")
    print("\nKesimpulan:", kes)

    # ---- Laporan PDF
    L = Laporan(os.path.join(BASE, "Lab2_Indeks_dan_Pencarian.pdf"), "Lab 2 - Bangun Indeks & Cari",
                f"ukuran chunk {UKURAN}, overlap {int(OVERLAP * 100)}%, backend: {idx.backend}")
    L.h1("Langkah 1. Membangun indeks")
    L.p("Tiga PDF diletakkan di folder dokumen/. Setiap file melaporkan jumlah chunk-nya:")
    L.tabel([["File", "Jumlah chunk"]] + [[f, n] for f, n in per_file.items()] + [["Total",
    len(idx.chunks)]],
            [9 * 28.35, 4 * 28.35])
    L.kode(log_indeks.rstrip())
    L.h1("Langkah 2. Lima pertanyaan dan pemeriksaan chunk teratas")
    L.p("Sebuah chunk dianggap relevan bila memuat semua kata kunci jawaban. Tabel menunjukkan 3 chunk
    teratas.")
    for s, atas, rel in h2:
        L.h2(s["q"])
        L.tabel([["Peringkat", "Skor", "Sumber", "Relevan?", "Cuplikan"]] +
                [[r, f"{a['skor']:.3f}", f"{a['file']} #{a['no']}", "ya" if ok else "tidak",
    ringkas(a["teks"], 140)]
                 for r, (a, ok) in enumerate(zip(atas, rel), 1)],
                [1.6 * 28.35, 1.3 * 28.35, 3.8 * 28.35, 1.6 * 28.35, 8.7 * 28.35])
    L.p(f"Ringkasan: chunk peringkat 1 relevan pada {n_top1}/5 pertanyaan; ada chunk relevan di 3 teratas
     pada {n_top3}/5.")
    L.h1("Langkah 3. Pertanyaan berparafrase")
    L.p("Dua pertanyaan memakai kata yang berbeda dari teks dokumen. Dibandingkan: pencarian semantik "
        f"({idx.backend}) vs TF-IDF.")
    baris = [["Pertanyaan", "Catatan", f"{idx.backend}: top1 / top3", "TF-IDF: top1 / top3"]]
    for (s, a1, r1), (_, a2, r2) in zip(sem, lek):
        baris.append([s["q"], s["catatan"], f"{'ya' if r1[0] else 'tidak'} / {'ya' if any(r1) else
    'tidak'}",
                      f"{'ya' if r2[0] else 'tidak'} / {'ya' if any(r2) else 'tidak'}"])
    L.tabel(baris, [5 * 28.35, 6 * 28.35, 3 * 28.35, 3 * 28.35])
    for (s, a1, r1), _ in zip(sem, lek):
        L.p(f"Chunk teratas untuk \"{s['q']}\": {a1[0]['file']} #{a1[0]['no']} (skor
    {a1[0]['skor']:.3f})")
        L.kode(ringkas(a1[0]["teks"], 400))
    L.h1("Kesimpulan")
    L.p(kes)
    L.halaman_baru()
    L.h1("Lampiran: kode lab2_indeks_cari.py")
    L.kode(open(__file__, encoding="utf-8").read())
    L.simpan()
    print("\nLaporan: Lab2_Indeks_dan_Pencarian.pdf")


if __name__ == "__main__":
```

```
main()
```